Evaluating Mixture-of-Recursions Blocks in Transformer-Based Object Detectors: Query Initialization Determines When Shared Recursive Depth Pays Off

Abstract

Mixture-of-Recursions (MoR) unifies two efficiency mechanisms in one Transformer: weights shared across depth, and a learned router that assigns each token its own number of recursions. It has proven effective in language models and vision backbones, but its behavior inside detection transformers, where the routed tokens are object queries trained by one-to-one set matching, has not been studied. We evaluate MoR blocks in two detector families that differ chiefly in how decoder queries are initialized: DETR, whose queries are input-independent embeddings, and RT-DETR, whose queries are selected from encoder features together with reference boxes. All models use an ImageNet-initialized ResNet-34 backbone and are trained on PASCAL VOC, and each MoR model matches the maximum effective depth of its dense baseline. In RT-DETR, two shared decoder blocks applied three times reach 66.03 average precision (AP), against 65.31 for a six-layer decoder, with 65% fewer decoder parameters at matched decoder compute; three blocks applied twice reach 66.10. Both variants improve 16 of 20 classes, with the largest gains at strict overlap thresholds. In DETR, MoR halves the transformer parameters but loses 1.50 to 4.07 AP. Inference-time probes that remove, randomize or oracle-guide the routing change AP by at most 0.62 in any model, so the deficit is not a routing error, and exit-depth analysis shows that recursion refines well-initialized queries but does not repair poorly initialized ones. We derive practical guidelines for using recursive depth in detection transformers.

Keywords: Object detection, Detection transformer, Mixture-of-Recursions, Parameter sharing, Adaptive computation, Query initialization

1 Introduction

Detection transformers cast object detection as direct set prediction: a decoder refines a fixed set of object queries through a stack of attention layers, and a bipartite matching loss assigns each query to at most one object [7]. Decoder depth is therefore a central design variable. Each layer re-estimates every query's class and box, and successive layers progressively correct earlier estimates [7, 45]. Real-time variants such as RT-DETR [42] expose the number of decoder layers as a speed–accuracy knob, and every additional layer adds its own set of weights.

Two families of techniques reduce the cost of Transformer depth. *Parameter sharing* reuses one block several times, as in Universal Transformers [8], ALBERT [18] and recursive Transformers [3]. *Adaptive computation* lets each token use only as much depth as it needs, through halting [6, 12], early exits [34] or learned routing [29, 44]. Mixture-of-Recursions (MoR) [4] combines the two. A shared stack of layers is applied recursively, and a lightweight router decides, token by token, how many recursions each token receives. MoR improved the accuracy–efficiency trade-off of language models at equal compute [4], and it has since been adapted to vision backbones [1, 2, 14, 19, 22].

Whether these benefits carry over to the encoder–decoder of a detection transformer is not obvious. Decoder tokens are not image patches but object queries, and each query must be refined into exactly one box or into "no object".

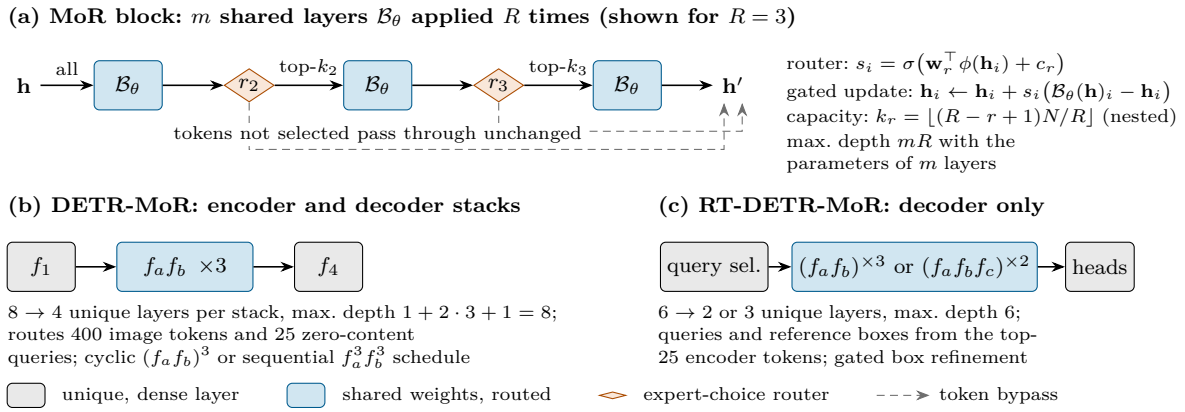


Fig. 1 Mixture-of-Recursions (MoR) in detection transformers. (a) A block of m shared layers is applied R times. Before every repeated pass, an expert-choice router keeps the top-k_r tokens, which receive a gated residual update, and the remaining tokens pass through unchanged. (b) In DETR, the middle layers of both the encoder and the decoder are shared and routed, keeping the maximum effective depth of eight. (c) In RT-DETR, the whole decoder is a recursive block applied to the queries produced by query selection, keeping the maximum effective depth of six

A recursive block applies the *same* transformation repeatedly. That suits an iterative-refinement process in which every step makes a small correction to a hypothesis that is already close to its target, a view formalized by equilibrium formulations of the decoder [5, 35]. It is less clear that one shared transformation can also *form* object hypotheses from uninformative starting points. Efficient DETR [36] found that the many decoder layers of DETR are needed mainly because its object queries are randomly initialized, and that a dense prior removes most of this need. Detectors with poor and with informed query initialization may therefore respond very differently to recursive depth.

This paper evaluates MoR blocks in the two regimes. DETR [7] starts every decoder query from a zero content vector and a learned, input-independent positional embedding. RT-DETR [42] selects its queries from the highest-scoring encoder tokens and attaches a reference box to each, which every decoder layer then refines. We insert MoR into both detectors, keep the backbone, data and training recipe of each family fixed, and match every MoR model to the *maximum effective depth* of its dense baseline. Our contributions are as follows.

- We present a controlled evaluation of MoR blocks inside the encoder–decoder of detection transformers, rather than in the backbone, covering two detector families that represent the two query-initialization regimes, two parameter-sharing schedules and two block/recursion splits.
- In RT-DETR, the two MoR decoders outperform the six-layer decoder of equal maximum depth by 0.71 and 0.78 AP with 65% and 49% fewer decoder parameters at matched decoder FLOPs. The 2×3 decoder outperforms the two-layer decoder with the same unique parameters by 1.29 AP. The gains concentrate at strict IoU thresholds and hold for 16 of 20 classes.
- In DETR, MoR halves the transformer parameters but loses 1.50 (cyclic) and 4.07 (sequential) AP. Inference-time routing probes show that this deficit does not come from routing decisions: removing the routing altogether leaves it in place.
- An analysis of what the routers allocate shows that recursion acts as refinement of well-initialized queries in RT-DETR but cannot repair poorly initialized queries in DETR. We distill this into practical guidelines.

2 Related work

Detection transformers and query initialization.

DETR [7] removed anchors and non-maximum suppression through Hungarian matching [17] but converges slowly. Deformable DETR [45] introduced sparse multi-scale deformable attention,

Table 1 Configurations compared. "Max depth" is the maximum effective depth of the decoder (and of the encoder for DETR). "Avg. depth" is the average number of layer applications per query (per encoder token for DETR) implied by the capacity schedule

Model	Routed stacks	Unique layers	$m \times R$	Max depth	Avg. depth	Decoder capacity
RT-DETR (2L)	–	2	–	2	2	–
RT-DETR (6L)	–	6	–	6	6	–
RT-DETR-MoR (2×3)	decoder	2	2×3	6	3.92	25→16→8
RT-DETR-MoR (3×2)	decoder	3	3×2	6	4.44	25→12
DETR	–	8 / 8	–	8	8 / 8	–
DETR-MoR (cyclic)	enc. + dec.	4 / 4	$1{+}2 \times 3{+}1$	8	6.00 / 5.92	25→16→8
DETR-MoR (sequential)	enc. + dec.	4 / 4	$1{+}2 \times 3{+}1$	8	5.50 / 5.40	25→20→…→4

iterative box refinement and a two-stage variant that initializes queries from encoder proposals. Later work made queries more informative: conditional spatial queries [26], anchor boxes as queries [23], query denoising [20], and mixed query selection [39]. Efficient DETR [36] traced DETR's need for many decoder layers to the random initialization of its object queries and reference points, and with a dense prior closed the gap between one and six decoder layers. RT-DETR [42] combines an efficient hybrid encoder with IoU-aware query selection to reach real-time speed, and RT-DETRv2 [25], D-FINE [27] and DEIM [16] refine its training.

Weight sharing and recursion.

Universal Transformers [8] and ALBERT [18] share weights across depth, looped Transformers apply one block repeatedly [11], and relaxed recursive Transformers relax strict tying with layer-wise low-rank adapters [3]. In detection, RecursiveDet [41] shares decoder parameters across the stages of region-based detectors, and DEQDet [35] models the decoder as a fixed-point iteration of a weight-tied layer, following deep equilibrium models [5].

Adaptive computation and routing.

Adaptive computation time [12], PonderNet [6] and early-exit networks [34] vary depth per input. Sparse mixtures of experts [10, 33] route tokens to experts, expert-choice routing [44] lets each expert select a fixed number of tokens, and Mixture-of-Depths [29] uses that mechanism to let tokens skip whole blocks. In vision, DynamicViT [28] and A-ViT [37] prune or halt tokens. For DETR encoders, Sparse DETR [31], Focus-DETR [43] and Salience DETR [15] restrict computation to selected tokens, and Lite DETR [21] updates low-level tokens less often. For decoders, QR-DETR [32] routes queries through or around each layer, and Dual-R-DETR [40] routes query-to-query interactions in self-attention. None of these methods shares weights across depth.

Mixture-of-Recursions in vision.

MoR [4] combines a recursive Transformer with token-level routing and recursion-wise key–value caching. Vision-MoR [14], MoR–Swin [2], MoR-ST [1], Edge-RecViT [22] and soft MoR [19] apply recursion with adaptive depth inside image *backbones*, and some of them report detection results with conventional heads. To our knowledge, no prior study places MoR inside the encoder–decoder of a detection transformer or relates its benefit to how the object queries are initialized. That is the gap this work addresses.

3 Method

3.1 Decoder preliminaries

A detection-transformer decoder maps N queries $\mathbf{Q}^{(0)} \in \mathbb{R}^{N \times d}$ through layers $f_1, \ldots, f_L$, $\mathbf{Q}^{(\ell)} = f_\ell(\mathbf{Q}^{(\ell-1)}, \mathbf{M})$. Each layer consists of query self-attention, cross-attention to the encoder memory $\mathbf{M}$, and a feed-forward network. Every layer output is decoded by class and box heads and supervised through Hungarian matching. The two detectors differ in $\mathbf{Q}^{(0)}$:

- In DETR [7], the content part of each query is zero and the positional part is a learned embedding that does not depend on the image. Boxes are predicted from scratch at every layer.
- In RT-DETR [42], the N encoder tokens with the highest class scores become the content queries, each with a reference box $\mathbf{b}^{(0)}$. Every layer predicts a correction, $\mathbf{b}^{(\ell)} =$

$\sigma\big(\sigma^{-1}(\mathbf{b}^{(\ell-1)}) + \Delta\mathbf{b}^{(\ell)}\big)$, and cross-attention is deformable [45].

3.2 Mixture-of-Recursions block

A MoR block consists of m unique layers $\mathcal{B}_\theta = (f_1, \ldots, f_m)$ that are applied R times with the same weights. Its maximum effective depth is therefore mR, while its parameter count is that of m layers. Following the expert-choice variant of MoR [4, 44], every recursion $r \geq 2$ has its own router, a single linear layer with sigmoid output:

$$s_i^{(r)} = \sigma\big(\mathbf{w}_r^\top \phi(\mathbf{h}_i) + c_r\big), \tag{1}$$

where ϕ is a LayerNorm in RT-DETR-MoR and the identity in DETR-MoR. The first recursion is dense. Recursion r keeps the k_r highest-scoring tokens of the previously active set,

$$\begin{aligned} \mathcal{A}^{(r)} &= \text{top-}k_r\big\{s_i^{(r)} : i \in \mathcal{A}^{(r-1)}\big\}, \\ k_r &= \max\Big(1, \Big\lfloor \tfrac{R-r+1}{R}\, N \Big\rfloor\Big), \end{aligned} \tag{2}$$

so the active set can only shrink, and N queries give capacities $25 \to 16 \to 8$ for $R = 3$ and $25 \to 12$ for $R = 2$. Active tokens are updated by a gated residual and inactive tokens pass through unchanged:

$$\mathbf{h}_i \leftarrow \begin{cases} \mathbf{h}_i + s_i^{(r)}\big(\mathcal{B}_\theta(\mathbf{h})_i - \mathbf{h}_i\big), & i \in \mathcal{A}^{(r)}, \\ \mathbf{h}_i, & \text{otherwise.} \end{cases} \tag{3}$$

Self-attention inside a routed recursion runs only among active tokens, and cross-attention always sees the full memory. Because the top-k selection is not differentiable, the router is trained only through its gate (3). We add no auxiliary router loss, so routing is learned purely from the detection objective.

3.3 Integration into the two detectors

Figure 1 shows both integrations, and Table 1 summarizes the configurations.

DETR-MoR (encoder and decoder).

Both stacks of the eight-layer DETR follow a middle-cycle layout. A unique first layer and a unique last layer process all tokens, and between them two shared layers are applied $R = 3$ times under routing. The maximum effective depth is $1 + 2 \cdot 3 + 1 = 8$, equal to the baseline, with four unique layers per stack instead of eight. The encoder routes the 400 image tokens ($400 \to 266 \to 133$) and the decoder routes the 25 queries. We compare two sharing schedules:

- *cyclic*: the order is $(f_a, f_b) \times 3$, with one routing decision before each repeated pass;
- *sequential*: the order is $f_a, f_a, f_a, f_b, f_b, f_b$, with a decision before every application, giving five routers per stack and decoder capacities $25 \to 20 \to 16 \to 12 \to 8 \to 4$.

RT-DETR-MoR (decoder only).

The backbone, hybrid encoder and query selection are unchanged. The decoder is a pure cycle of m shared layers applied R times, with $m \times R \in \{2 \times 3, 3 \times 2\}$, so the maximum effective depth is six, equal to the six-layer baseline. The gate also scales the box correction, $\mathbf{b}_i \leftarrow \sigma\big(\sigma^{-1}(\mathbf{b}_i) + s_i^{(r)}\Delta\mathbf{b}_i\big)$, so a query that is barely selected is barely moved. Each recursion has its own classification head and is supervised, while the box heads are shared. Denoising queries [20] are never routed.

3.4 Routing probes

To test whether the learned routing is responsible for a model's accuracy, we change the routing *at inference only*, on checkpoints trained with routing, and re-evaluate.

- *Full*: every token is kept at every recursion, which removes routing. The learned gates are still applied.
- *Random*: tokens are kept at random under the trained capacities.
- *Oracle*: decoder only. Queries that the Hungarian matcher assigns to ground-truth objects in the default pass get priority under the trained capacities.

The *full* probe is out of distribution for a routed checkpoint. A rise in AP would therefore indict the router, whereas a drop would prove little.

4 Experimental setup

Data.

We use PASCAL VOC [9].

Table 2 RT-DETR family on VOC2007 test. Decoder parameters include the per-layer heads. GFLOPs are for one 640×640 image with 25 queries

	Params (M)		GFLOPs					
Model	Decoder	Total	Decoder	Total	AP	AP_{50}	AP_{75}	AP_{85}
RT-DETR (2L)	2.27	34.54	2.32	110.0	64.74	85.76	70.74	52.85
RT-DETR (6L)	6.54	38.81	6.95	114.6	65.31	86.58	70.58	53.41
RT-DETR-MoR (2×3)	2.27	34.54	6.83	114.5	66.03	**86.82**	71.84	54.72
RT-DETR-MoR (3×2)	3.33	35.60	6.86	114.5	**66.10**	86.35	**72.06**	**55.21**
RT-DETR-MoR (2×3) − RT-DETR (6L)					+0.71	+0.25	+1.27	+1.31
RT-DETR-MoR (3×2) − RT-DETR (6L)					+0.78	−0.23	+1.48	+1.80
RT-DETR-MoR (2×3) − RT-DETR (2L) (same unique parameters)					+1.29	+1.07	+1.11	+1.88

Table 3 DETR family on VOC2007 test. All three models share an identical training recipe. GFLOPs are for one 640×640 image

	Params (M)		GFLOPs				
Model	Transformer	Trainable	Transformer	Total	AP	AP_{50}	AP_{75}
DETR	23.15	23.43	11.20	71.2	**47.18**	**74.27**	**49.20**
DETR-MoR (cyclic)	11.58	11.85	8.45	68.4	45.69	73.03	47.80
DETR-MoR (sequential)	11.58	11.85	7.79	67.8	43.12	69.98	44.36
DETR-MoR (cyclic) − DETR					−1.50	−1.24	−1.40
DETR-MoR (sequential) − DETR					−4.07	−4.29	−4.84

- *Training pool*: VOC2007 and VOC2012 trainval (16,551 images). Because every detector uses 25 queries, the 15 images with more than 25 annotated objects are removed.
- *Split*: the remaining 16,536 images are split at random (seed 1111) into 13,229 training images and 3,307 validation images. The validation images are used only to select the checkpoint with the lowest validation loss.
- *Test*: VOC2007 test (4,952 images), with the same filter applied, leaving 4,945 images.

Detectors and training.

Both families use a ResNet-34 [13] initialized from ImageNet. Neither loads detection-pretrained weights, so every transformer, router and head is trained from scratch. Images are resized to 640×640. Training uses horizontal flips, zoom-out, IoU-constrained cropping and photometric distortion, and the seed is 1111.

DETR family.

- Architecture: the backbone, including batch normalization, is frozen. Pre-norm Transformer with $d = 256$, 8 heads, feed-forward width 2048, eight encoder and eight decoder layers, and 25 queries over the 20×20 feature map.
- Loss: cross-entropy with background weight 0.1, plus 5× L1 and 2× generalized-IoU [30] box losses at every decoder output.
- Optimization: AdamW [24], learning rate 1.4×10^{-4}, weight decay 10^{-4}, batch size 32, 300 epochs, with the learning rate reduced tenfold at epoch 200.
- The three DETR-family models use *identical* training settings.

RT-DETR family.

- Architecture: frozen batch-norm statistics, with backbone learning rate 10^{-5}. One AIFI layer on the stride-32 features, a CCFM fusion neck [42], top-25 query selection, and deformable cross-attention (8 heads, 3 levels, 4 points) with feed-forward width 1024.
- Loss: 50 denoising queries [20], varifocal loss [38] ($\alpha = 0.75$, $\gamma = 2$), plus 5× L1 and 2× generalized-IoU losses on every decoder output and on the query-selection output.
- Optimization: AdamW, learning rate 10^{-4}, weight decay 10^{-4}, gradient clipping at 0.1, 72 epochs, with the learning rate reduced tenfold at epoch 60.

- Batch size, set by the memory of a single 16 GB GPU: 23 (2L), 25 (6L), 20 (MoR 2×3) and 22 (MoR 3×2).

Evaluation.

We report average precision (AP) averaged over IoU thresholds $\{0.50, 0.55, \ldots, 0.95\}$, together with AP_{50} and AP_{75}.

- Per-class AP uses all-point interpolation of the precision–recall curve.
- Objects marked *difficult* are excluded both from the recall denominator and from scoring.
- No score threshold or non-maximum suppression is applied. DETR outputs one detection per query (25 per image), and RT-DETR keeps its top 100 query–class pairs.

The protocol is applied identically to all models, but it is not the COCO 101-point implementation, so absolute values should be compared only within this study.

Complexity.

We report parameters and floating-point operations (FLOPs, two per multiply–accumulate) for a single 640×640 image. FLOPs are counted with PyTorch's operator counter, with attention products added analytically and RepVGG branches fused. MoR FLOPs are counted as executed under the trained capacity schedule. We do not report latency, because the routed implementations are not kernel-optimized and a latency comparison would mostly measure implementation overhead.

Use of generative AI.

A large language model was used to help draft the text of this manuscript and the analysis scripts that compile its tables and figures from the stored evaluation outputs. The authors verified every number, statement and reference and take full responsibility for the content.

5 Results

5.1 RT-DETR: recursion improves accuracy at matched depth

Table 2 reports the RT-DETR family.

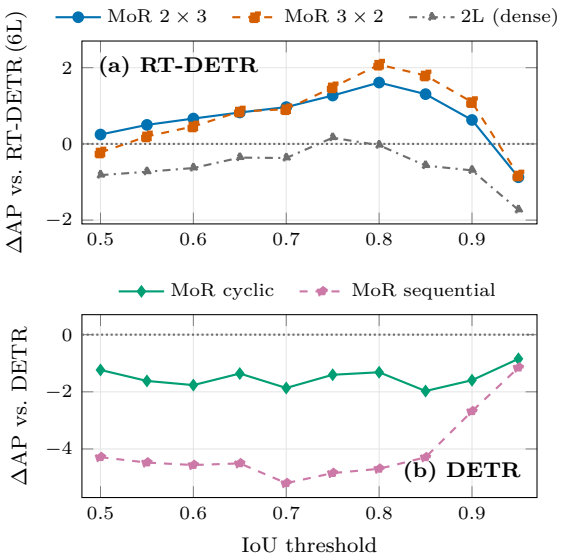


Fig. 2 AP difference to the dense baseline at each IoU threshold. (a) In RT-DETR, the gains of MoR grow with localization strictness and peak at IoU 0.80–0.85. (b) In DETR, the deficit of MoR is roughly uniform across thresholds, so it is not a localization effect

- Both MoR decoders outperform the six-layer decoder of equal maximum depth: RT-DETR-MoR (2×3) by 0.71 AP and RT-DETR-MoR (3×2) by 0.78 AP.
- They do so with 65% and 49% fewer decoder parameters, and the whole detector shrinks by 11.0% and 8.3%.
- The cleanest comparison is with RT-DETR (2L), which has the same unique decoder parameters as RT-DETR-MoR (2×3). Applying those two layers three times instead of once adds 1.29 AP. Adding four unshared layers (RT-DETR (6L)) adds only 0.58 AP.

In this regime, reusing a small decoder recursively was a better use of depth than stacking new layers.

Where the gain comes from.

Figure 2a breaks the difference down by IoU threshold.

- The gain grows with localization strictness: RT-DETR-MoR (2×3) improves on RT-DETR (6L) by +0.97 to +1.61 AP at IoU 0.70–0.85, and RT-DETR-MoR (3×2) by +0.91 to +2.08 AP.
- At IoU 0.50 the two configurations differ from the baseline by only +0.25 and −0.23.

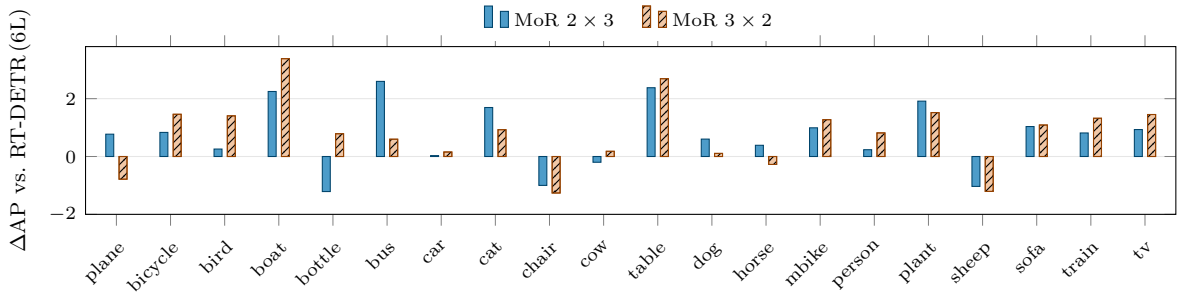


Fig. 3 Per-class AP difference between the RT-DETR-MoR decoders and the six-layer decoder of equal maximum depth. Each configuration improves 16 of the 20 classes

- At the extreme threshold of 0.95, both are slightly below it (−0.87, −0.83).

The recursive decoders therefore find essentially the same objects as the six-layer decoder but place their boxes more precisely. This fits the gated, shared box-refinement step of Section 3.3. The improvement is also not confined to a few categories: both configurations beat RT-DETR (6L) on 16 of the 20 classes, and RT-DETR-MoR (2×3) beats RT-DETR (2L) on 19 (Fig. 3).

Compute.

The decoder FLOPs of the MoR models (6.83 and 6.86 G) match those of RT-DETR (6L) (6.95 G), as intended by depth matching. Most of this cost is not query processing. 6.61 G of it is the value projection of the deformable attention over the 8,400 memory tokens, which is recomputed at every block application. Because the memory is fixed and the weights are shared, this projection is identical across recursions of the same block. Caching it once per unique block would reduce the MoR decoders to an estimated 2.42 and 3.55 G. This is the detection analogue of MoR's recursion-wise key–value caching [4], which our implementation does not yet exploit.

5.2 DETR: recursion does not compensate for poor initialization

Table 3 reports the DETR family.

- MoR halves the trainable parameters (49% fewer) and reduces the transformer FLOPs by 25% (cyclic) and 30% (sequential).
- Accuracy falls by 1.50 AP (cyclic) and 4.07 AP (sequential), and the drop is spread across classes: the cyclic model is worse on 18 of 20 and the sequential model on all 20.
- Unlike in RT-DETR, the deficit is nearly constant across IoU thresholds (Fig. 2b). For the cyclic model it lies between 0.85 and 1.97 AP at every threshold.

The flat deficit means that MoR loses objects and class confidence, not box precision. The schedule matters too: the sequential schedule makes five routing decisions and sends only 12 of 25 queries into its second shared layer, and it loses almost three times as much as the cyclic one.

5.3 Is the router responsible?

A natural explanation for the DETR result is that routing decisions made on poorly initialized queries are unreliable. If so, the router would withhold depth from the queries that need it. Table 4 rules this out at inference time. On the first 1,000 test images (2,924 matched objects), no probe moves AP by more than 0.62 in any model.

- *Removing routing does not help*: running every query and token at full depth changes the cyclic DETR-MoR by −0.19 AP, against a deficit of 1.50. Removing it only in the encoder or only in the decoder gives −0.04 and −0.05.
- *Guaranteeing depth to matched queries does not help*: the oracle probe gives −0.36.
- *Random routing* changes AP by only −0.62, the largest effect of any probe.
- The sequential model behaves the same way, and in RT-DETR-MoR all probes stay within 0.17 AP.

In both families, *which* queries recurse at inference is nearly irrelevant. The DETR deficit therefore lies in what the shared weights learned during

Table 4 Inference-time routing probes on checkpoints trained with routing (first 1,000 VOC2007 test images). AP under the trained routing, and its change under each probe; *full* removes routing, and *oracle* prioritizes the queries matched to ground truth

Model	AP	Full	Random	Oracle
DETR-MoR, cyc.	47.70	−0.19	−0.62	−0.36
DETR-MoR, seq.	45.17	+0.20	−0.22	−0.20
RT-DETR-MoR, 2 × 3	67.33	+0.14	−0.05	−0.07
RT-DETR-MoR, 3 × 2	67.14	+0.10	−0.17	−0.11

training, not in the routing decisions made at inference.

5.4 What recursion is spent on

DETR-MoR (cyclic) and RT-DETR-MoR (2×3) form a matched pair. Both apply two shared blocks three times, with decoder capacities 25→16→8. Figure 4 shows, for each matched query, the recursion after which it exited and the IoU of its final box.

- *DETR-MoR.* The router is not arbitrary. Queries that recurse beyond the first pass come from crowded images (7.3 and 7.9 objects on average, against 4.1) and end with much lower IoU (0.60 and 0.57, against 0.80). The router sends the hard queries deeper, but the extra recursions do not lift them to the quality of the queries that exit early.
- *RT-DETR-MoR.* Queries exiting after one or two recursions reach the same IoU (0.84 and 0.84), and only the 7% that use all three are lower (0.76). The queries enter the decoder already close to their objects, and recursion polishes them, which is consistent with the strict-IoU gains of Fig. 2a.

The encoder routers of DETR-MoR show a related pattern. The cyclic router keeps 53% of foreground tokens but only 17% of background tokens at its last stage, a 3.14× preference for objects that it learned without any supervision. The sequential router is not object-aware: its foreground-to-background ratio is 0.71–1.42. It is also the weaker model, which suggests that object-aware allocation of encoder recursion during training matters, even though forcing full depth at inference does not recover the loss.

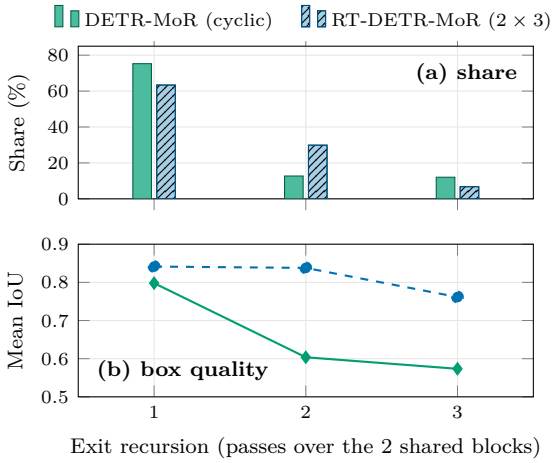


Fig. 4 Matched pair with identical recursion structure (2 shared blocks × 3 recursions, capacity 25→16→8), on the first 1,000 test images. (a) Share of matched queries by exit recursion. (b) Mean IoU of their final boxes. DETR-MoR sends hard queries deeper but cannot repair them, whereas RT-DETR-MoR queries are already accurate and are refined

6 Discussion

Why query initialization decides the outcome.

A weight-tied recursion applies one mapping repeatedly. It is well suited to a process in which every step makes a small correction to an estimate that is already near its target, the setting in which decoders behave like fixed-point iterations [35].

- *RT-DETR.* Query selection supplies exactly this setting: content features and reference boxes taken from the encoder. Our measurements show the signature of refinement. The gains are concentrated at strict IoU thresholds, the box quality barely depends on exit depth, and the routing decisions are almost interchangeable.
- *DETR.* The decoder starts from input-independent queries, so its early layers must form object hypotheses and suppress duplicates through self-attention before later layers can refine them. DETR's AP improves by several points from its first to its last decoder layer [7], and Efficient DETR traced the need for this depth to the random initialization of the queries [36]. Forcing one set of shared weights to perform both the formation and the refinement stages appears to cost accuracy across all IoU

thresholds. Our probes indicate that no routing policy recovers this capacity: the bottleneck is what the shared weights can represent, not which queries receive them.

Query initialization is the explanation that our evidence supports best, but it is not the only difference between the two settings:

- DETR-MoR also routes encoder tokens, whereas RT-DETR-MoR routes only decoder queries;
- DETR-MoR has 49% fewer trainable parameters than its baseline;
- the DETR backbone is frozen.

The routing probes and the matched-pair analysis, which isolates the detector family under identical recursion structures, make the initialization account the most consistent one.

Practical guidelines.

1. Use MoR in decoders whose queries are initialized from the image, through query selection, anchor priors or denoising. In our experiments it reduced decoder parameters and improved accuracy.
2. Prefer cyclic sharing with few routing decisions over sequential schedules that starve later shared layers.
3. Do not expect inference savings from a depth-matched recursive decoder unless the memory projections of shared blocks are cached.
4. In detectors with input-independent queries, keep the early layers unique or initialize the queries from the encoder before sharing depth. Improving the router alone is unlikely to help.

Limitations.

- *Single seed.* Each configuration was trained once. Two independently trained MoR configurations agree in sign, per-class pattern and IoU profile, but seed variance is not quantified.
- *Training budgets.* Batch size varied with GPU memory, so RT-DETR-MoR (2×3) received about 25% more optimizer steps than RT-DETR (6L). In addition, both RT-DETR (6L) and RT-DETR-MoR (2×3) were still improving at the final epoch.
- *Benchmark and protocol.* All experiments use VOC at 640 × 640 with at most 25 objects per image and a custom all-point AP implementation.
- *Missing measurements and baselines.* We report FLOPs but no measured latency, and we did not train a parameter-matched four-layer DETR baseline.

Multi-seed runs, COCO-scale experiments, cached memory projections and latency measurements are the natural next steps.

7 Conclusion

We evaluated Mixture-of-Recursions blocks inside two detection transformers that differ in how their object queries are initialized. When queries come from encoder-based query selection, as in RT-DETR, a recursive decoder of two or three shared layers matches the maximum depth of a six-layer decoder and exceeds its accuracy with up to 65% fewer decoder parameters, mainly by localizing objects more precisely. When queries start from input-independent embeddings, as in DETR, the same mechanism halves the transformer parameters but costs accuracy. That cost persists when routing is removed, randomized or oracle-guided at inference, which points to the shared weights, not the router, as the limiting factor. Recursive depth is therefore an effective way to compress detection decoders, provided that the queries it refines are well initialized.

Statements and Declarations

Funding. Provided on the separate title page (double-blind review).

Competing interests. The authors declare that they have no competing interests.

Ethics approval and consent. Not applicable.

Data availability. PASCAL VOC is publicly available (http://host.robots.ox.ac.uk/pascal/VOC/). The evaluation outputs supporting the findings are available from the corresponding author on reasonable request.

Code availability. Code and configurations will be released upon acceptance. They are withheld during review for anonymity.

Author contributions. Provided on the separate title page.

References

[1] Ai Y, Gao T (2026) MoR-ST: Content-adaptive recursive computation for efficient vision transformers. The Visual Computer 42(11):509. https://doi.org/10.1007/s00371-026-04713-6

[2] Ai Y, Gao T, Lin Z, et al (2026) MoR–Swin: Efficient vision transformer using mixture of recursions. Journal of Imaging 12(8):377. https://doi.org/10.3390/jimaging12080377

[3] Bae S, Fisch A, Harutyunyan H, et al (2025) Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA. In: International Conference on Learning Representations (ICLR)

[4] Bae S, Kim Y, Bayat R, et al (2025) Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In: Advances in Neural Information Processing Systems, pp 96572–96617

[5] Bai S, Kolter JZ, Koltun V (2019) Deep equilibrium models. In: Advances in Neural Information Processing Systems, pp 688–699

[6] Banino A, Balaguer J, Blundell C (2021) PonderNet: Learning to ponder. arXiv preprint arXiv:2107.05407

[7] Carion N, Massa F, Synnaeve G, et al (2020) End-to-end object detection with transformers. In: Proceedings of the European Conference on Computer Vision (ECCV), Lecture Notes in Computer Science, vol 12346. Springer, Cham, pp 213–229, https://doi.org/10.1007/978-3-030-58452-8_13

[8] Dehghani M, Gouws S, Vinyals O, et al (2019) Universal transformers. In: International Conference on Learning Representations (ICLR)

[9] Everingham M, Van Gool L, Williams CKI, et al (2010) The PASCAL visual object classes (VOC) challenge. International Journal of Computer Vision 88(2):303–338. https://doi.org/10.1007/s11263-009-0275-4

[10] Fedus W, Zoph B, Shazeer N (2022) Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research 23(120):1–39

[11] Giannou A, Rajput S, Sohn JY, et al (2023) Looped transformers as programmable computers. In: Proceedings of the 40th International Conference on Machine Learning (ICML), pp 11398–11442

[12] Graves A (2016) Adaptive computation time for recurrent neural networks. arXiv preprint arXiv:1603.08983

[13] He K, Zhang X, Ren S, et al (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp 770–778, https://doi.org/10.1109/CVPR.2016.90

[14] He Y, Yuan Z, Sun W, et al (2026) VisionMoR: Scaling vision transformer via patch-level mixture-of-recursions. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 4699–4707, https://doi.org/10.1609/aaai.v40i6.42471

[15] Hou X, Liu M, Zhang S, et al (2024) Salience DETR: Enhancing detection transformer with hierarchical salience filtering refinement. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 17574–17583, https://doi.org/10.1109/CVPR52733.2024.01664

[16] Huang S, Lu Z, Cun X, et al (2025) DEIM: DETR with improved matching for fast convergence. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 15162–15171, https://doi.org/10.1109/CVPR52734.2025.01412

[17] Kuhn HW (1955) The Hungarian method for the assignment problem. Naval Research Logistics Quarterly 2(1–2):83–97. https://

doi.org/10.1002/nav.3800020109

[18] Lan Z, Chen M, Goodman S, et al (2020) ALBERT: A lite BERT for self-supervised learning of language representations. In: International Conference on Learning Representations (ICLR)

[19] Lee SI, Park J (2026) Soft mixture-of-recursions: Going deeper with recursive vision transformers. arXiv preprint arXiv:2607.00774

[20] Li F, Zhang H, Liu S, et al (2022) DN-DETR: Accelerate DETR training by introducing query DeNoising. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 13609–13617, https://doi.org/10.1109/CVPR52688.2022.01325

[21] Li F, Zeng A, Liu S, et al (2023) Lite DETR: An interleaved multi-scale encoder for efficient DETR. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 18558–18567, https://doi.org/10.1109/CVPR52729.2023.01780

[22] Li Y, Xu J, Yin M, et al (2026) Edge-RecViT: Efficient vision transformer via semantic-refined dynamic recursion. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 12987–12996

[23] Liu S, Li F, Zhang H, et al (2022) DAB-DETR: Dynamic anchor boxes are better queries for DETR. In: International Conference on Learning Representations (ICLR)

[24] Loshchilov I, Hutter F (2019) Decoupled weight decay regularization. In: International Conference on Learning Representations (ICLR)

[25] Lv W, Zhao Y, Chang Q, et al (2024) RT-DETRv2: Improved baseline with bag-of-freebies for real-time detection transformer. arXiv preprint arXiv:2407.17140

[26] Meng D, Chen X, Fan Z, et al (2021) Conditional DETR for fast training convergence. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp 3631–3640, https://doi.org/10.1109/ICCV48922.2021.00363

[27] Peng Y, Li H, Wu P, et al (2025) D-FINE: Redefine regression task of DETRs as fine-grained distribution refinement. In: International Conference on Learning Representations (ICLR)

[28] Rao Y, Zhao W, Liu B, et al (2021) DynamicViT: Efficient vision transformers with dynamic token sparsification. In: Advances in Neural Information Processing Systems, pp 13937–13949

[29] Raposo D, Ritter S, Richards B, et al (2024) Mixture-of-depths: Dynamically allocating compute in transformer-based language models. arXiv preprint arXiv:2404.02258

[30] Rezatofighi H, Tsoi N, Gwak J, et al (2019) Generalized intersection over union: A metric and a loss for bounding box regression. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 658–666, https://doi.org/10.1109/CVPR.2019.00075

[31] Roh B, Shin J, Shin W, et al (2022) Sparse DETR: Efficient end-to-end object detection with learnable sparsity. In: International Conference on Learning Representations (ICLR)

[32] Senthivel T, Vu NS (2024) QR-DETR: Query routing for detection transformer. In: Proceedings of the Asian Conference on Computer Vision (ACCV), Lecture Notes in Computer Science, vol 15477. Springer, Singapore, pp 395–412, https://doi.org/10.1007/978-981-96-0960-4_24

[33] Shazeer N, Mirhoseini A, Maziarz K, et al (2017) Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In: International Conference on Learning Representations (ICLR)

[34] Teerapittayanon S, McDanel B, Kung HT (2016) BranchyNet: Fast inference via early exiting from deep neural networks. In: Proceedings of the 23rd International Conference on Pattern Recognition (ICPR), pp 2464–2469, https://doi.org/10.1109/ICPR.2016.7900006

[35] Wang S, Teng Y, Wang L (2023) Deep equilibrium object detection. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp 6273–6283, https://doi.org/10.1109/ICCV51070.2023.00579

[36] Yao Z, Ai J, Li B, et al (2021) Efficient DETR: Improving end-to-end object detector with dense prior. arXiv preprint arXiv:2104.01318

[37] Yin H, Vahdat A, Alvarez JM, et al (2022) A-ViT: Adaptive tokens for efficient vision transformer. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 10799–10808, https://doi.org/10.1109/CVPR52688.2022.01054

[38] Zhang H, Wang Y, Dayoub F, et al (2021) VarifocalNet: An IoU-aware dense object detector. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 8510–8519, https://doi.org/10.1109/CVPR46437.2021.00841

[39] Zhang H, Li F, Liu S, et al (2023) DINO: DETR with improved DeNoising anchor boxes for end-to-end object detection. In: International Conference on Learning Representations (ICLR)

[40] Zhang Y, Chen Q, Huang W, et al (2025) Dual-R-DETR: Resolving query competition with pairwise routing in transformer decoders. arXiv preprint arXiv:2512.13876

[41] Zhao J, Sun L, Li Q (2023) RecursiveDet: End-to-end region-based recursive object detection. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp 6284–6293, https://doi.org/10.1109/ICCV51070.2023.00580

[42] Zhao Y, Lv W, Xu S, et al (2024) DETRs beat YOLOs on real-time object detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 16965–16974, https://doi.org/10.1109/CVPR52733.2024.01605

[43] Zheng D, Dong W, Hu H, et al (2023) Less is more: Focus attention for efficient DETR. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp 6651–6660, https://doi.org/10.1109/ICCV51070.2023.00614

[44] Zhou Y, Lei T, Liu H, et al (2022) Mixture-of-experts with expert choice routing. In: Advances in Neural Information Processing Systems, pp 7103–7114

[45] Zhu X, Su W, Lu L, et al (2021) Deformable DETR: Deformable transformers for end-to-end object detection. In: International Conference on Learning Representations (ICLR)